

Paging vs. Segmentation – Performance Evaluation

Fundamentals of Operating System Design

Prashant Jadav

DEPSTAR - CHARUSAT

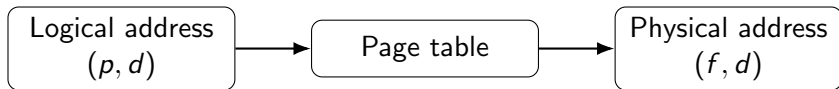
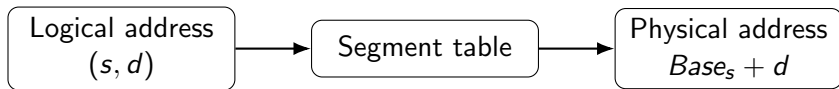
Learning Objectives

- Compare paging and segmentation from a performance perspective.
- Analyze address-translation overhead.
- Calculate memory-access time for paging and segmentation.
- Explain the role of the TLB.
- Quantify page-table and segment-table memory overhead.
- Compare fragmentation, locality, protection and sharing.
- Evaluate scalability and workload-dependent trade-offs.
- Solve numerical and GATE-style performance questions.

What Does Performance Evaluation Mean?

Metric	Question
Translation latency	How many memory accesses are needed?
Table overhead	How much memory is consumed by mapping structures?
Fragmentation	How much memory is wasted?
Locality	How well can caches/TLB exploit access locality?
Protection	How naturally can access rights be enforced?
Sharing	How easily can code/data be shared?
Scalability	How does overhead grow with address-space size?
Allocation	How easily can memory be allocated and reclaimed?

Paging vs. Segmentation: Architectural Difference



Segmentation: variable-size logical units

Paging: fixed-size pages/frames

Segmentation:

$(s, d) \rightarrow \text{segment-table lookup} \rightarrow \text{bounds check} \rightarrow PA$

Paging:

$(p, d) \rightarrow \text{page-table lookup} \rightarrow (f, d)$

Without a TLB, a page-table lookup can require an additional memory access.

Key point

The actual performance depends on the hardware implementation, descriptor/page-table caching, and TLB behavior.

Single-Level Paging Without TLB

Let:

T_M = memory access time

For one memory reference:

- 1 Access the page table.
- 2 Access the requested data.

Hence:

$$EAT = 2T_M$$

For:

$$T_M = 100 \text{ ns}$$

$$EAT = 2(100) = 200 \text{ ns}$$

Thus page-table lookup can approximately double the memory-reference cost if no translation cache is available.

Paging with TLB

Let:

h = TLB hit ratio, T_T = TLB lookup time, T_M = memory access time

TLB hit:

$$T_{\text{hit}} = T_T + T_M$$

TLB miss:

$$T_{\text{miss}} = T_T + 2T_M$$

Therefore:

$$EAT = h(T_T + T_M) + (1 - h)(T_T + 2T_M)$$

or:

$$EAT = T_T + T_M + (1 - h)T_M$$

Numerical Example: TLB

Given:

$$T_T = 10 \text{ ns}, \quad T_M = 100 \text{ ns}, \quad h = 0.90$$

Hit:

$$10 + 100 = 110 \text{ ns}$$

Miss:

$$10 + 200 = 210 \text{ ns}$$

$$\begin{aligned} EAT &= 0.9(110) + 0.1(210) \\ &= \boxed{120 \text{ ns}} \end{aligned}$$

Observation

A high TLB hit ratio substantially reduces the effective translation cost of paging.

Segmentation Access-Time Example

Assume:

$$T_M = 100 \text{ ns}$$

If the segment descriptor is already available in a hardware register:

$$EAT \approx 100 \text{ ns}$$

If the descriptor must first be fetched from memory:

$$EAT \approx 2T_M = 200 \text{ ns}$$

Important

Segmentation is not automatically faster than paging. Actual performance depends on how segment descriptors and page translations are cached and accessed.

Multi-Level Paging: Translation Cost

For a k -level page table, without a TLB:

$$EAT = (k + 1) T_M$$

Paging structure	Memory accesses
1-level	2
2-level	3
3-level	4
4-level	5

Multi-level paging reduces page-table memory requirements for sparse address spaces, but a TLB miss may require multiple table accesses.

Performance Metric: Mapping-Table Memory

Paging:

Page-table size = number of pages $\times$ PTE size

Segmentation:

Segment-table size = number of segments $\times$ segment-entry size

Paging overhead depends strongly on page count

Segmentation overhead depends strongly on segment count

Numerical Example: Page-Table Overhead

Given:

32-bit virtual address, 4 KB pages, 4 B PTE

Offset bits:

$$\log_2(4096) = 12$$

Virtual pages:

$$2^{32-12} = 2^{20}$$

Page-table size:

$$2^{20} \times 4 = 2^{22} = \boxed{4 \text{ MB}}$$

Performance implication

A large page table consumes memory and can create additional cache and management overhead.

Numerical Example: Segment-Table Overhead

Suppose:

64 segments

and each entry is:

16 B

Then:

$$64 \times 16 = \boxed{1024 \text{ B}}$$

If 100 processes have the same structure:

$$100 \times 1024 = \boxed{100 \text{ KB}}$$

Observation

A small number of logical segments can make the segment-table footprint relatively small.

Performance Metric: Fragmentation

Paging

- No external fragmentation caused by fixed-size page/frame allocation.
- Internal fragmentation may occur, especially in the final page.

Segmentation

- Variable-size allocation can create external fragmentation.
- Compaction may be required.

Paging: internal fragmentation

Segmentation: external fragmentation

Numerical Example: Paging Fragmentation

Process size:

18.5 KB

Page size:

4 KB

Pages required:

$$\left\lceil \frac{18.5}{4} \right\rceil = 5$$

Allocated:

$$5 \times 4 = 20 \text{ KB}$$

Internal fragmentation:

$$20 - 18.5 = \boxed{1.5 \text{ KB}}$$

Numerical Example: Segmentation Fragmentation

Suppose free holes are:

10 KB, 6 KB, 8 KB

Total free memory:

$$10 + 6 + 8 = \boxed{24 \text{ KB}}$$

A request for:

20 KB

cannot be satisfied by a contiguous segment.

$\boxed{24 \text{ KB free does not imply a 20-KB contiguous region}}$

Paging avoids this particular contiguous-allocation requirement.

Segmentation

- Segment sizes vary.
- Contiguous placement can create external fragmentation.
- Compaction may be expensive.

Paging

- Pages can occupy arbitrary physical frames.
- A process need not be physically contiguous.
- Physical movement can be handled at page granularity.

Performance Metric: Protection

Segmentation naturally represents logical protection domains:

- code: read + execute,
- data: read + write,
- stack: read + write.

Paging can provide protection per page:

- read/write/execute permissions,
- user/supervisor permissions,
- shared pages.

Performance consideration

Finer-grained protection may require more metadata and more management work.

Performance Metric: Sharing

Segmentation: logical-unit sharing

Example:

shared code segment

Paging: fixed-size page sharing

Example:

shared library pages

Segmentation maps naturally to logical program components; paging permits sharing at page granularity.

Performance Metric: Locality

Paging

- TLB caches recent page translations.
- Virtual-memory systems exploit page locality.
- Sequential access can benefit from spatial locality.

Segmentation

- Segment descriptors can be cached by hardware.
- Logical segments can correspond naturally to working regions.

Key point

Locality is primarily a workload and hardware property; the memory-management scheme determines how that locality is represented and exploited.

Paging vs. Segmentation: Performance Matrix

Criterion	Paging	Segmentation
Translation	Page-table/TLB lookup	Segment lookup + bounds check
Physical allocation	Fixed-size frames	Variable-size regions
External fragmentation	Eliminated	Possible
Internal fragmentation	Possible	Generally lower at allocation-unit level
Table size	Can be large	Often smaller if segment count is small
Logical organization	Less direct	Strong
Protection	Page-level	Natural segment-level
Sharing	Page-level	Natural segment-level
Virtual memory	Very convenient	More complex alone
Compaction	Not needed for external fragmentation	May be required

Segmentation with Paging: Performance Trade-off

Logical address:

$$(s, p, d)$$

Translation:

segment table $\rightarrow$ page table $\rightarrow$ physical frame

Advantages:

- logical organization,
- segment-level protection/sharing,
- fixed-size physical allocation.

Costs:

- additional table structures,
- more translation complexity,
- more metadata,
- dependence on TLB performance.

Flexibility increases, but translation/storage complexity also increases

Worked Comparison: Access Time

Assume:

$$T_M = 100 \text{ ns}, \quad T_{TLB} = 10 \text{ ns}, \quad h = 95\%$$

Paging:

$$EAT = 10 + 100 + (1 - 0.95)(100) = \boxed{115 \text{ ns}}$$

If a segment descriptor is available in a hardware register:

$$EAT \approx \boxed{100 \text{ ns}}$$

If the descriptor requires an additional memory access:

$$EAT \approx \boxed{200 \text{ ns}}$$

Conclusion

Hardware support determines actual performance; the scheme name alone is insufficient.

Workload-Dependent Evaluation

- Large address spaces → page-table overhead becomes important.
- Many logical regions → segment metadata increases.
- Fragmented physical memory → variable-size contiguous allocation suffers.
- High translation frequency → TLB performance becomes critical.
- Shared libraries → sharing granularity matters.
- Strong logical protection boundaries → segmentation can represent them naturally.

Evaluate workload + hardware + memory hierarchy together

GATE-Style Numerical Question 1

A system has:

32-bit virtual addresses, 4 KB pages, 4 B PTE

Find the size of a single-level page table.

$$\text{offset} = 12 \text{ bits}$$

$$\text{pages} = 2^{20}$$

$$\text{PT size} = 2^{20} \times 4 = \boxed{4 \text{ MB}}$$

Performance issue: The mapping structure itself consumes significant memory.

GATE-Style Numerical Question 2

Given:

$$T_M = 100 \text{ ns}, \quad T_{TLB} = 10 \text{ ns}, \quad h = 90\%$$

$$EAT = 0.9(10 + 100) + 0.1(10 + 200)$$

$$= \boxed{120 \text{ ns}}$$

Exam pattern

Always calculate the TLB-hit and TLB-miss paths separately before applying probabilities.

GATE-Style HOT Question

A designer claims:

"Paging is always faster than segmentation because paging eliminates external fragmentation."

Critically evaluate the statement.

- Fragmentation and translation latency are different metrics.
- Paging may require page-table access.
- TLB misses can increase translation overhead.
- Segmentation may use cached descriptors.
- Paging can introduce internal fragmentation.
- Actual performance depends on hardware and workload.

A fragmentation advantage does not automatically imply an access-time advantage

Performance Evaluation Framework

Ask these questions:

- 1 What is the physical allocation granularity?
- 2 How many translation-table entries are required?
- 3 How many memory references occur during translation?
- 4 Is a TLB available and what is its hit ratio?
- 5 What type of fragmentation occurs?
- 6 How important are protection and sharing boundaries?
- 7 How large is the address space?
- 8 What is the workload's locality?

Performance = translation + memory + metadata + fragmentation effects

Summary

- Paging provides fixed-size allocation and eliminates external fragmentation.
- Segmentation provides a logical view and natural segment-level protection/sharing.
- Paging can incur page-table and TLB-related overhead.
- Segmentation can incur external fragmentation and compaction overhead.
- TLB hit ratio is a major factor in paging performance.
- Page-table size can become significant for large address spaces.
- No single metric determines which scheme has better performance.

Key takeaway

Performance must be evaluated using translation latency, memory overhead, fragmentation, locality, protection, sharing, hardware support, and workload characteristics.

$$\text{Segmentation: } PA = Base_s + Offset, \quad Offset < Limit_s$$

$$\text{Paging: } PA = Frame_p \times PageSize + Offset$$

$$\text{Pages} = 2^{VA \text{ bits} - \text{offset bits}}$$

$$\text{Page-table size} = \text{number of pages} \times \text{PTE size}$$

$$EAT_{TLB} = h(T_T + T_M) + (1 - h)(T_T + 2T_M)$$

$$EAT_{k\text{-level paging, no TLB}} = (k + 1)T_M$$